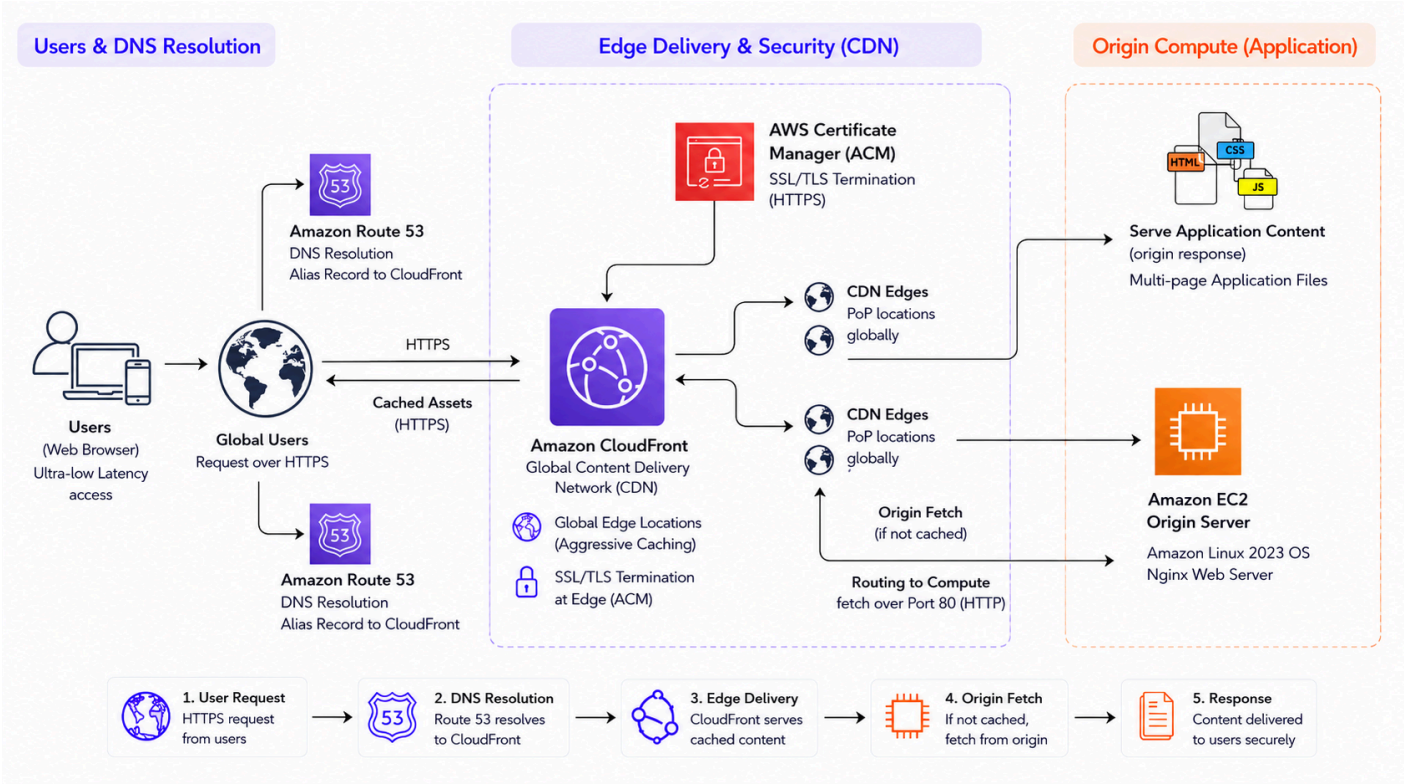
Global Content Distribution: Edge-Optimized Architecture

Objective: To architect and deploy a highly available, globally distributed delivery network for a multi-page frontend application, emphasizing edge caching, HTTPS enforcement, and strict origin security.

Solution: Implemented a decoupled content delivery architecture where a multi-page application (HTML/Tailwind CSS/JS) is hosted on an Amazon EC2 instance running Nginx server, set up as a CloudFront origin. Traffic is cached and routed globally via Amazon CloudFront, with custom DNS resolution managed by Amazon Route 53 and end-to-end TLS encryption provided by AWS Certificate Manager (ACM).



Phase 1: Compute & Web Server (Amazon EC2)
An EC2 instance (virtual server) is launched and configured to serve the frontend application files.

1) **EC2 Instance Details:**

- Name: myLinuxVM
- Amazon Machine Image (AMI): Amazon Linux 2023 kernel 6.1 AMI (Free tier eligible)
- Instance type: t3.micro (Free tier eligible)
- Key-pair name: mumbai-key
- Security group name: EC2-CloudFront-Origin-SG (custom created security group)

- Security group description: my custom security group
- Inbound Rule 1 (**SSH**):
    - Port: 22
    - Source type: **My IP** (strictly limited to admin access)
- Inbound Rule 2 (**HTTP**):
    - Port: 80
    - Source type: **Custom**
    - Source: com.amazonaws.global.cloudfront.origin-facing (**pl-9aa247f3**)

2) Application Deployment:

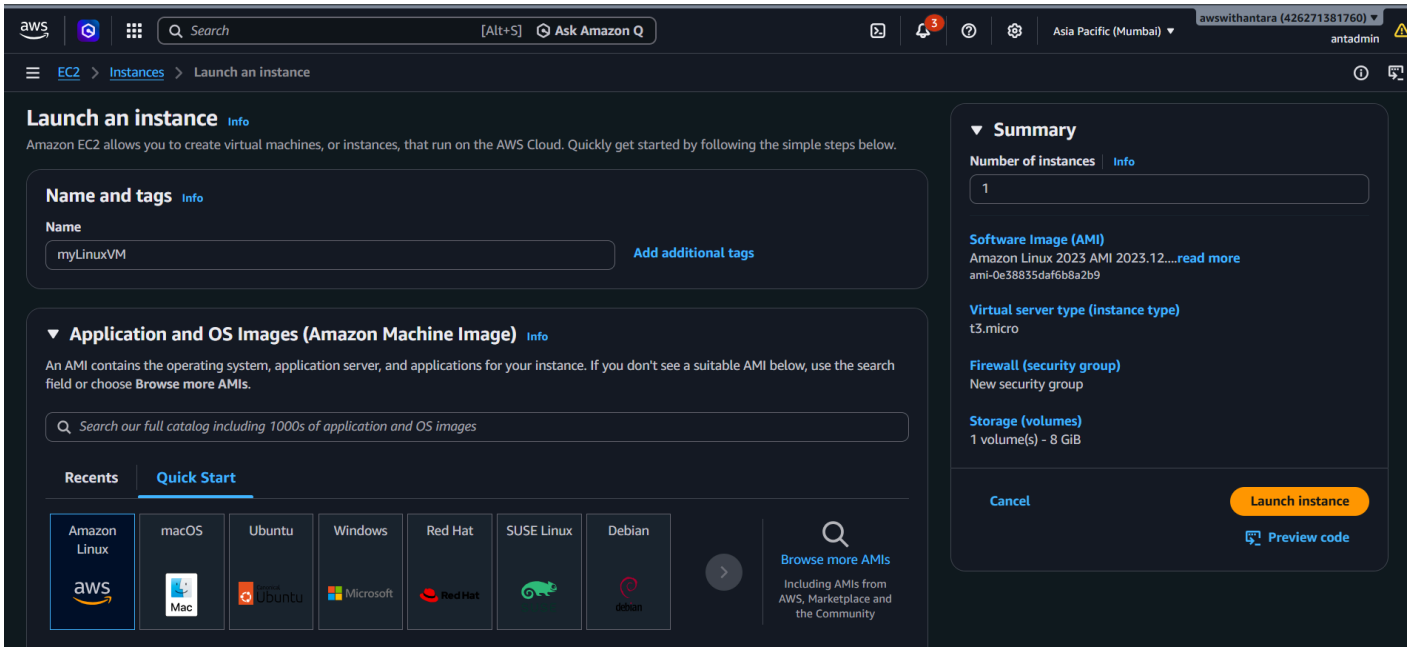
- EC2 instance (myLinuxVM) connected via SSH command using terminal
    - cd Downloads (location of the SSH Key)
    - ssh -i "mumbai-key.pem" ec2-user@ec2-65-0-102-31.ap-south-1.compute.amazonaws.com

- Linux commands to **install** and **enable** Nginx web server on **myLinuxVM**
    - sudo yum install nginx -y
    - sudo systemctl start nginx
    - sudo systemctl enable nginx

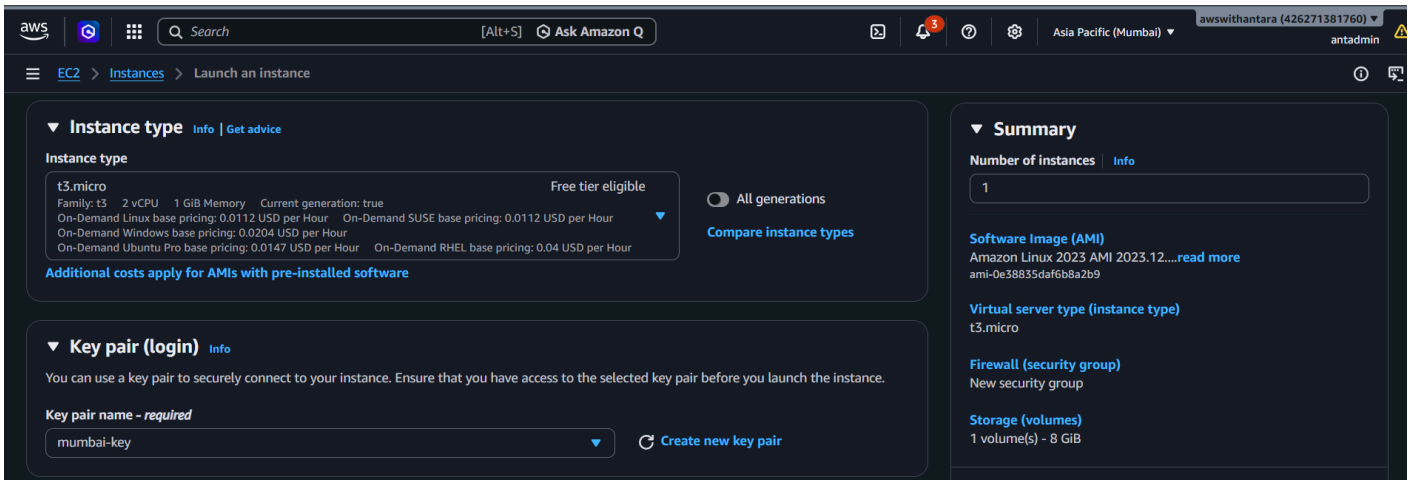
- Frontend files transferred from local machine to **myLinuxVM** using separate terminal
    - cd Downloads (location of the SSH Key and zipped Frontend files)
    - scp -i mumbai-key.pem Codespot_Project.zip ec2-user@65.0.102.31:/home/ec2-user

- Linux commands to deploy the application to **myLinuxVM** using the initial terminal where the web server is installed and enabled on the EC2 instance. These commands are used from the home directory of ec2-user, i.e., **/home/ec2-user**
    - ls (shows **Codespot_Project.zip** transferred to **/home/ec2-user**)
    - unzip **Codespot_Project.zip** (to unzip and extract files)
    - ls (shows both **Codespot_Project.zip** and **Codespot_Project** in **/home/ec2-user**)
    - sudo mv Codespot_Project/* /usr/share/nginx/html (to move all files from **/home/ec2-user** to **/usr/share/nginx/html**)

The **scp** command is used to securely transfer files from the local system to the EC2 instance. **/usr/share/nginx/html** is the location of the default html file in the Nginx web server.

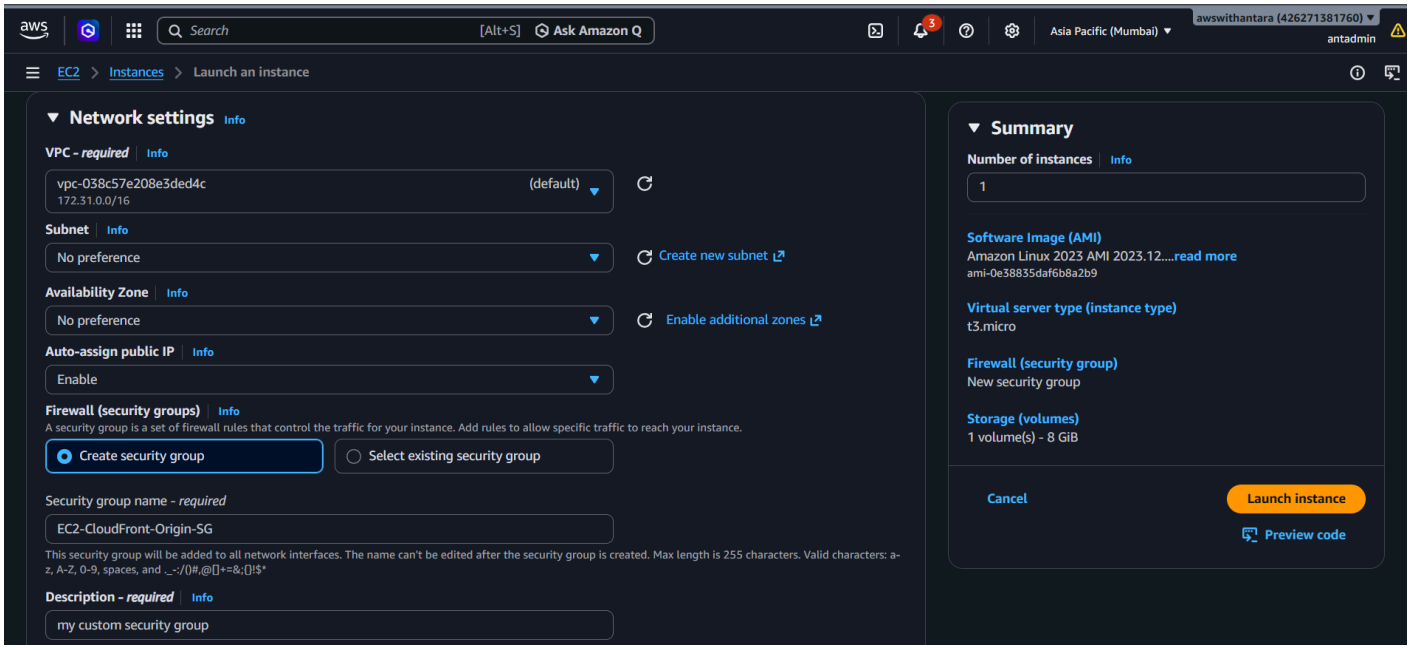


EC2 Instance Launch: Name & Operating System (AMI)

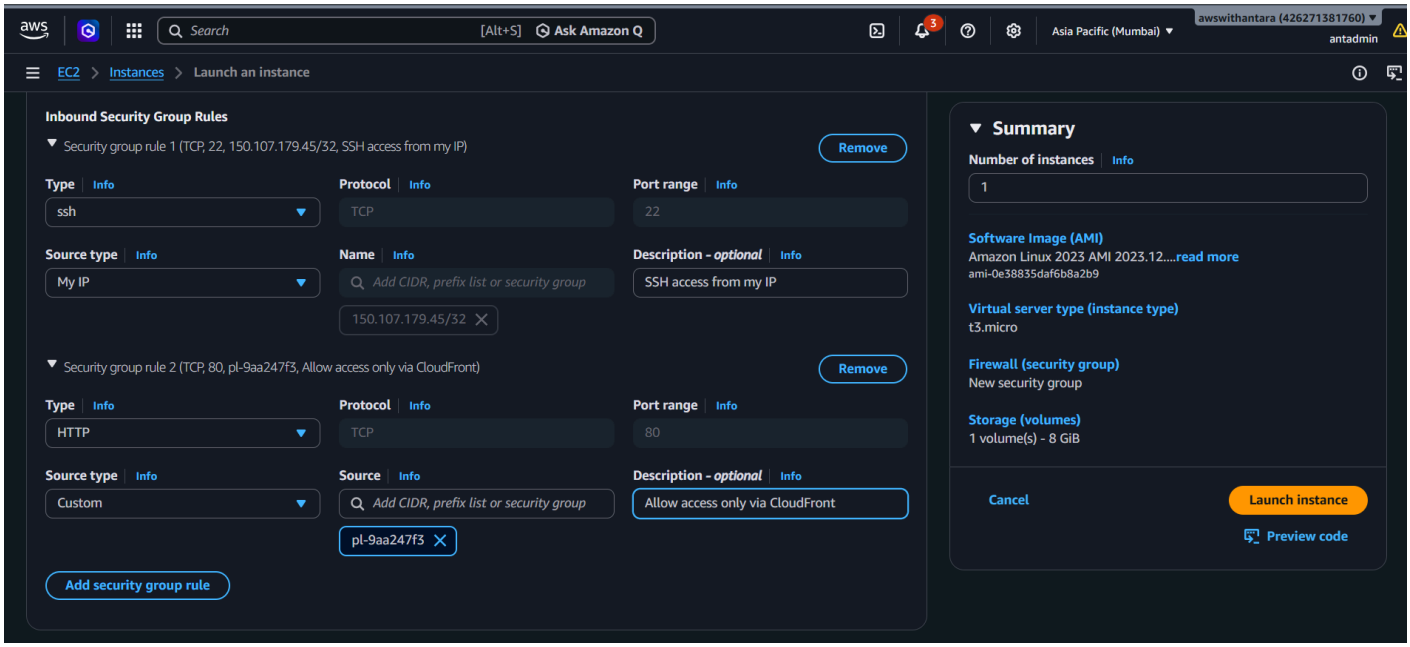


EC2 Instance Launch: Instance Type & Key Pair

- Operating System (AMI) & Instance type of the EC2 instance (**myLinuxVM**) are within the AWS Free tier, thereby ensuring cost optimization
- SSH Key Pair (mumbai-key) is essential for connecting to **myLinuxVM**



EC2 Instance Launch: VPC Settings, Security Group Name & Description



EC2 Instance Launch: Security Group Inbound Rules on Port 22 & 80

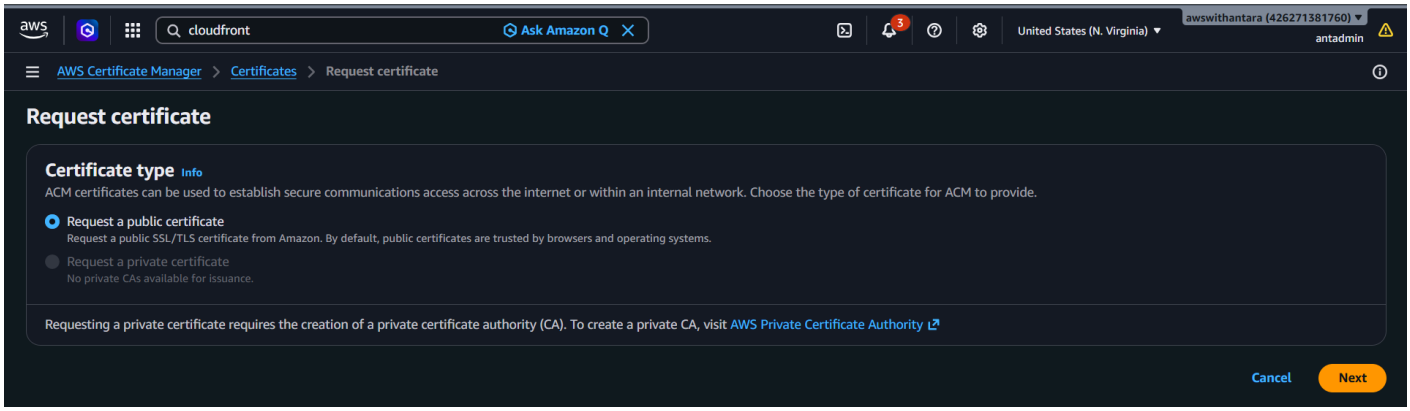
- VPC, Subnets, & Availability Zone are set to default settings for **myLinuxVM**
- Port 80 Inbound Rule has the source as **pl-9aa247f3** that relates to **com.amazonaws.global.cloudfront.origin-facing** which ensures that this virtual server is accessible only through CloudFront

Phase 2: Security & Encryption (Amazon Certificate Manager)
ACM is mandatory for ensuring security and encryption for the custom domain.

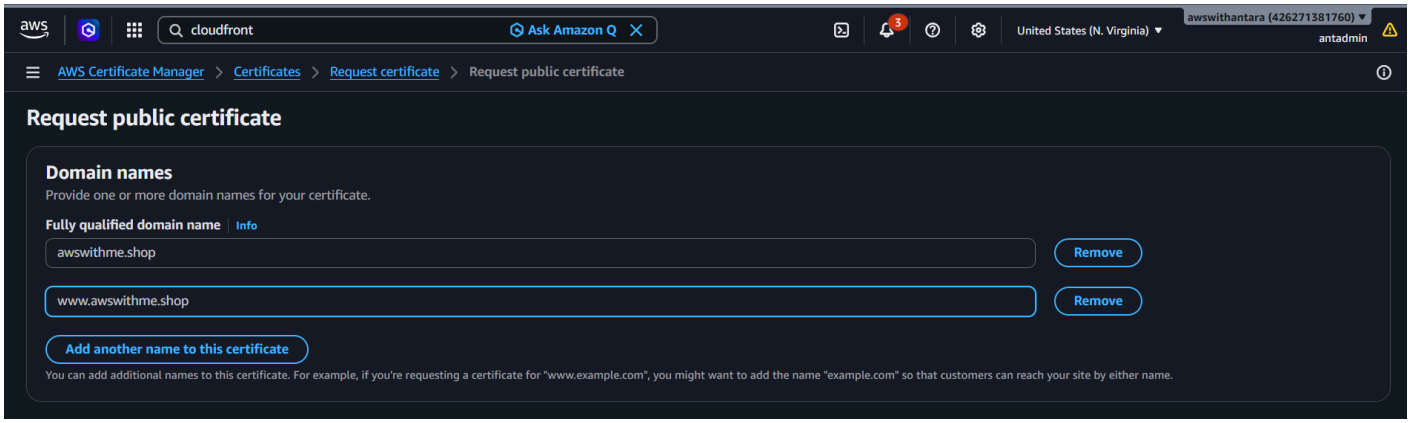
Certificate Details:

- Certificate type: Request a public certificate
- Domain names: awswithme.shop & www.awswithme.shop
- Validation method: DNS validation
- Key algorithm: RSA 2048 (default)

The certificate must be issued in **us-east-1** (North Virginia) because CloudFront's global edge network was designed to pull certificates exclusively from this region.

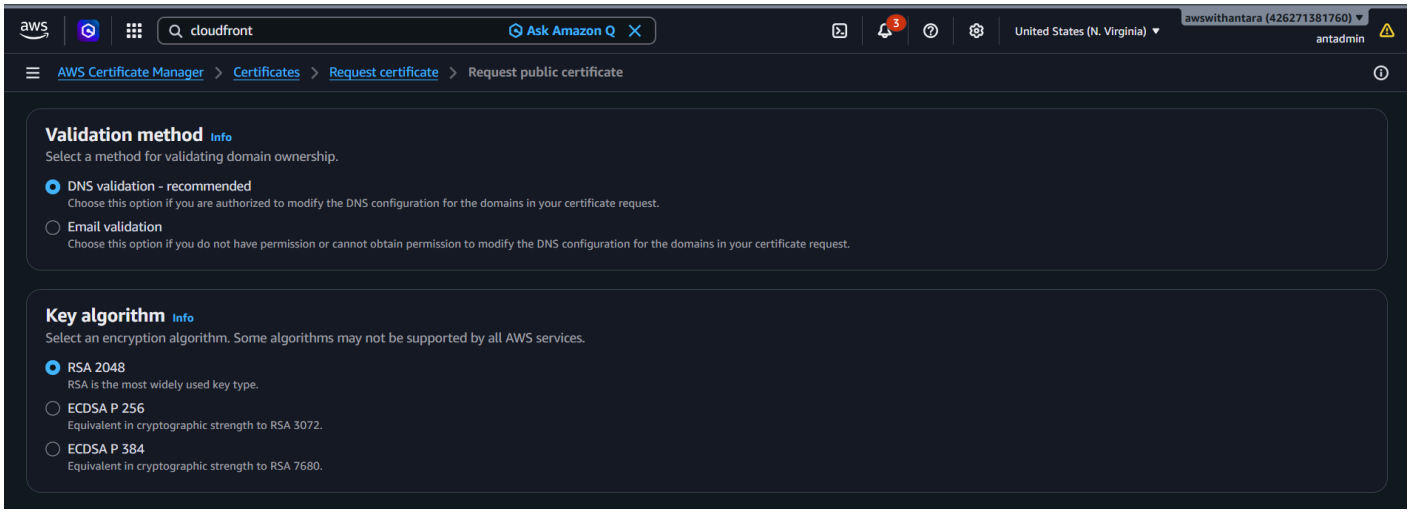


Issue Certificate in ACM: Certificate type

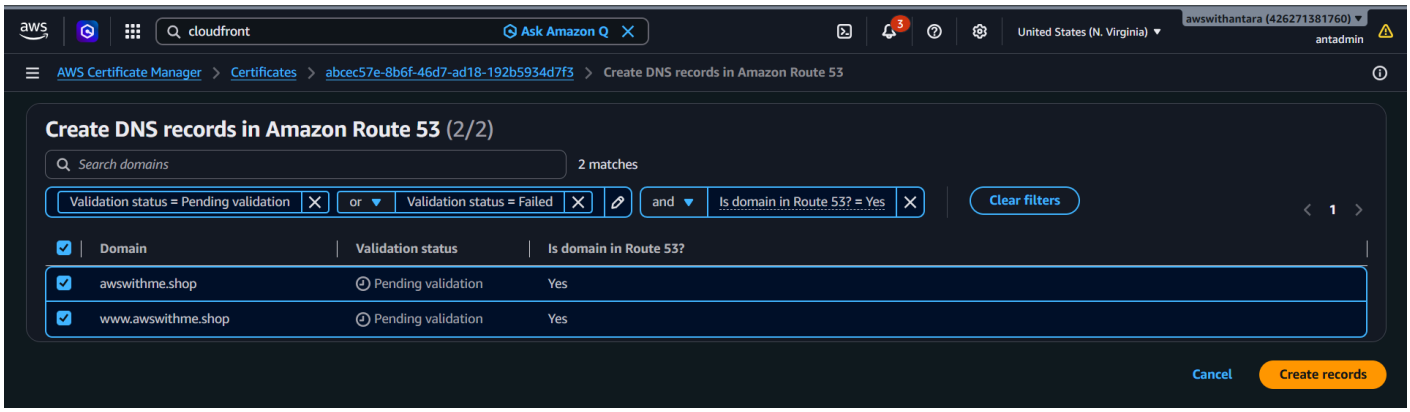


Issue Certificate in ACM: Domain Names

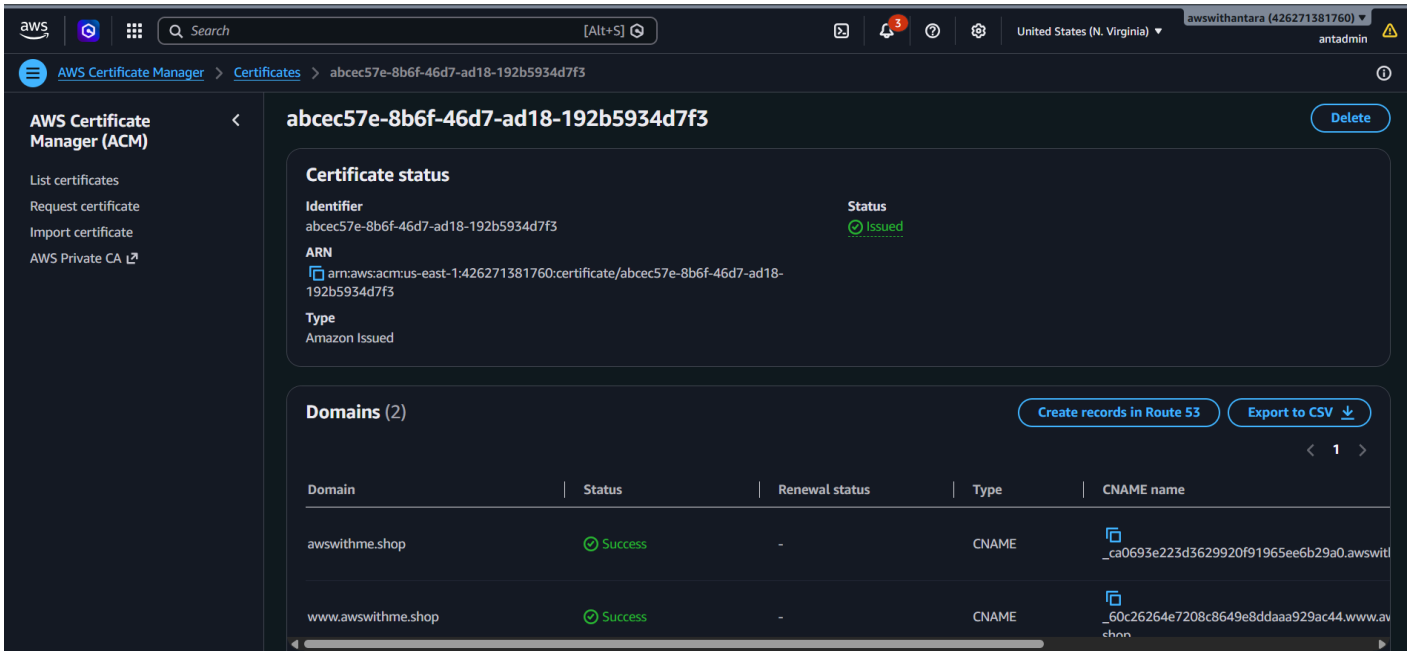
- Region is switched to **us-east-1 (N. Virginia)** in the ACM console since it's mandatory for CloudFront
- In addition to awswithme.shop, a subdomain is added to the certificate as required



Issue Certificate in ACM: Validation Method & Key Algorithm



Issue Certificate in ACM: DNS Records created in Route 53



Issue Certificate in ACM: Certificate Status

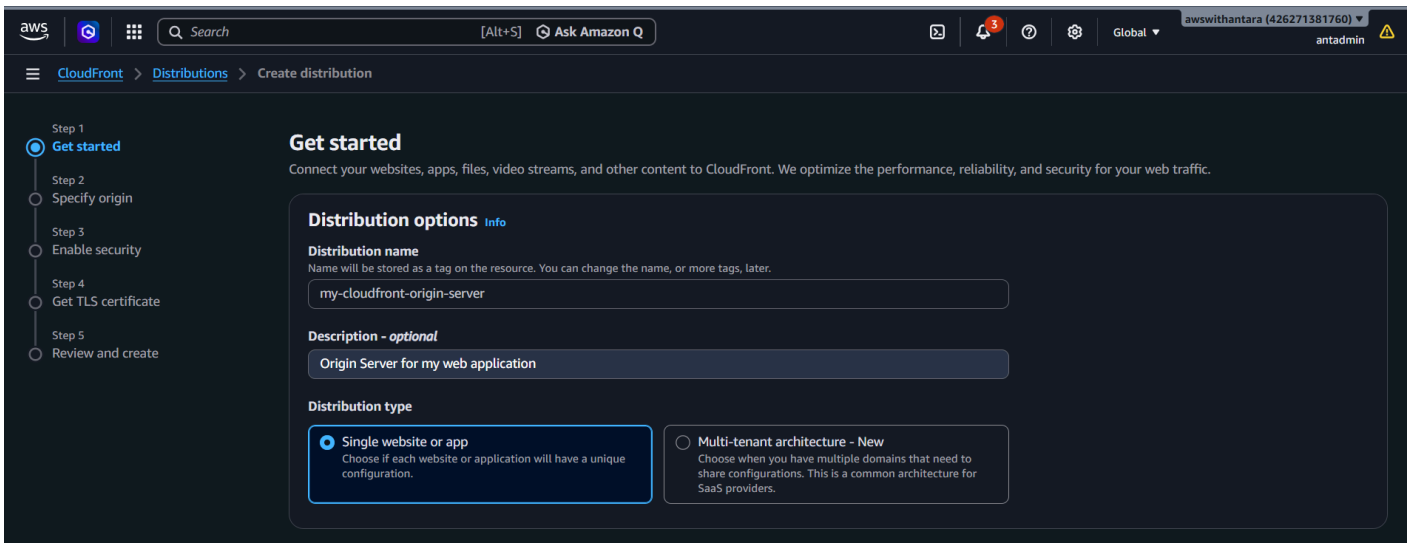
Phase 3: Global Content Delivery (Amazon CloudFront)
A CloudFront Distribution is created with EC2 origin to speed up content delivery through global edge caching.

1) **Distribution Details:**

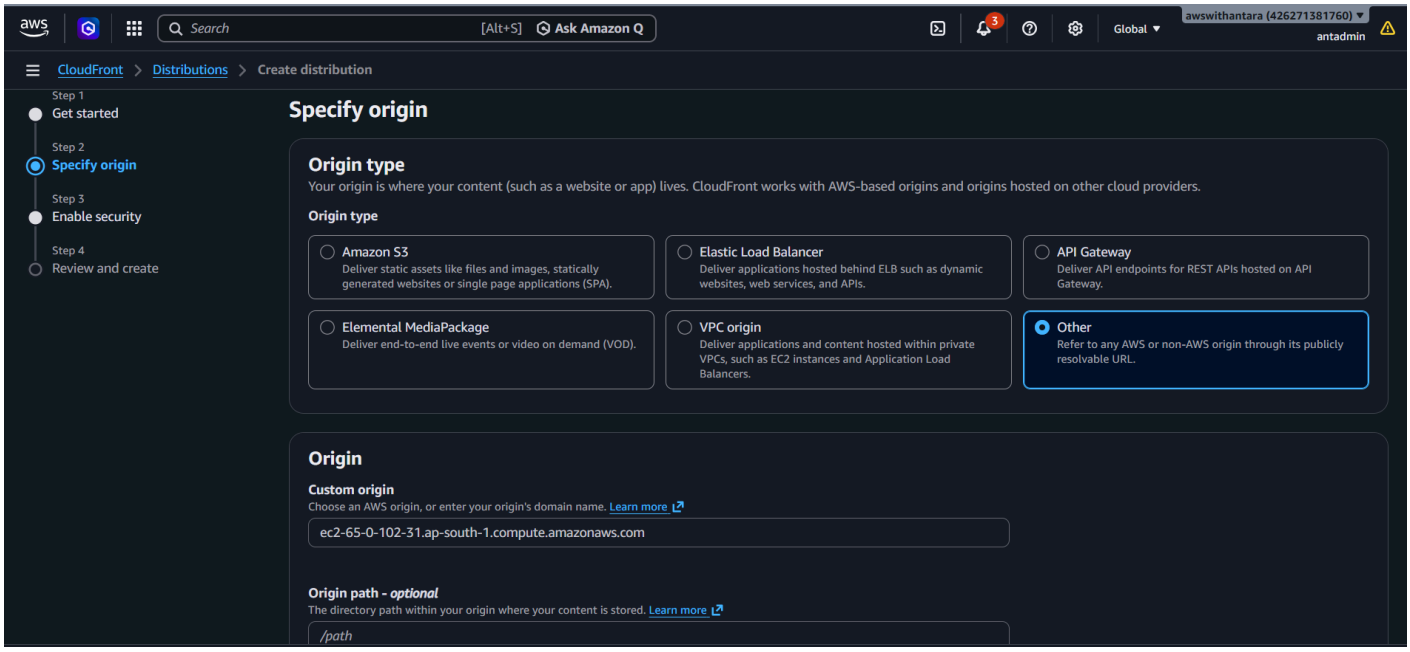
- Distribution name: my-cloudfront-origin-server
- Description: Origin Server for my web application
- Distribution type: Single website or app

2) **Origin Details:**

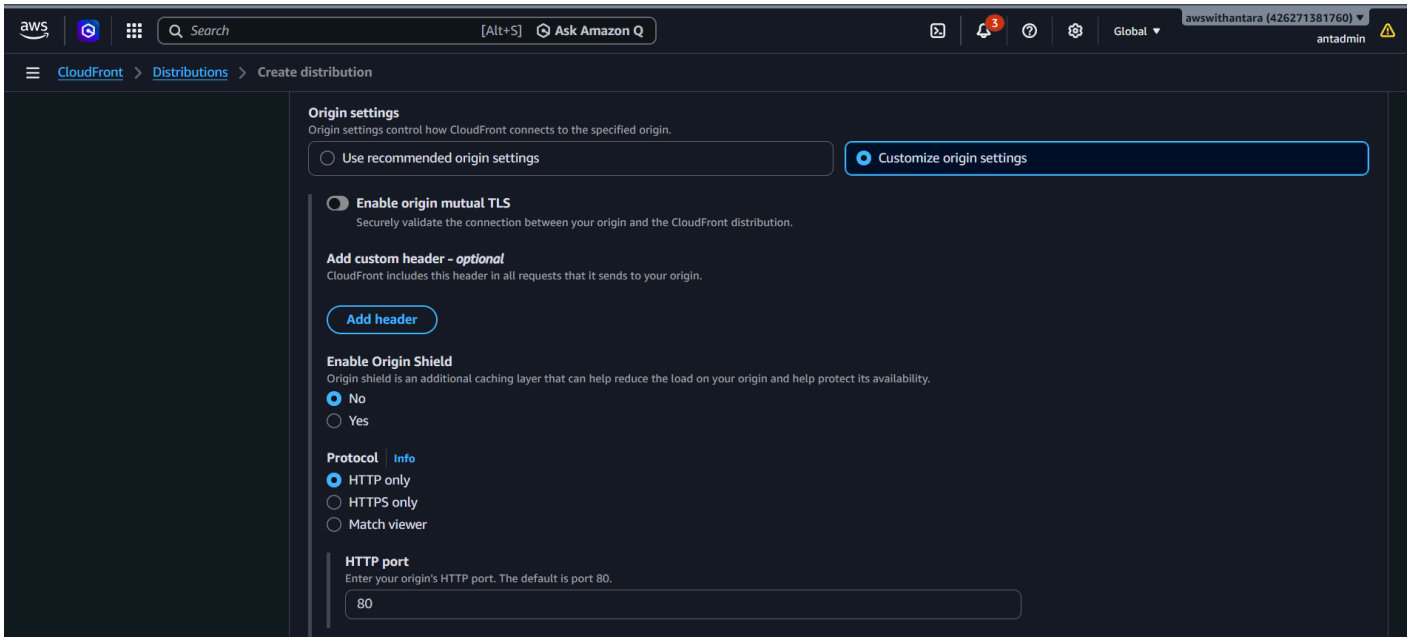
- Origin type: Other
- Custom origin: ec2-65-0-102-31.ap-south-1.compute.amazonaws.com (public DNS of the EC2 instance **myLinuxVM**)
- Origin settings: Customize origin settings
- Protocol: HTTP only
- Origin IP address type: IPv4-only
- Cache settings: Customize cache settings
- Viewer protocol policy: Redirect HTTP to HTTPS
- Allowed HTTP methods: GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE (default)
- Edit General Settings:
    - Alternate Domain Name (CNAME): awswithme.shop & www.awswithme.shop
    - Custom SSL certificate: awswithme.shop (**abcec57e-8b6f-46d7-ad18-192b5934d7f3**)



CloudFront Distribution Setup: Distribution Name, Description & Type

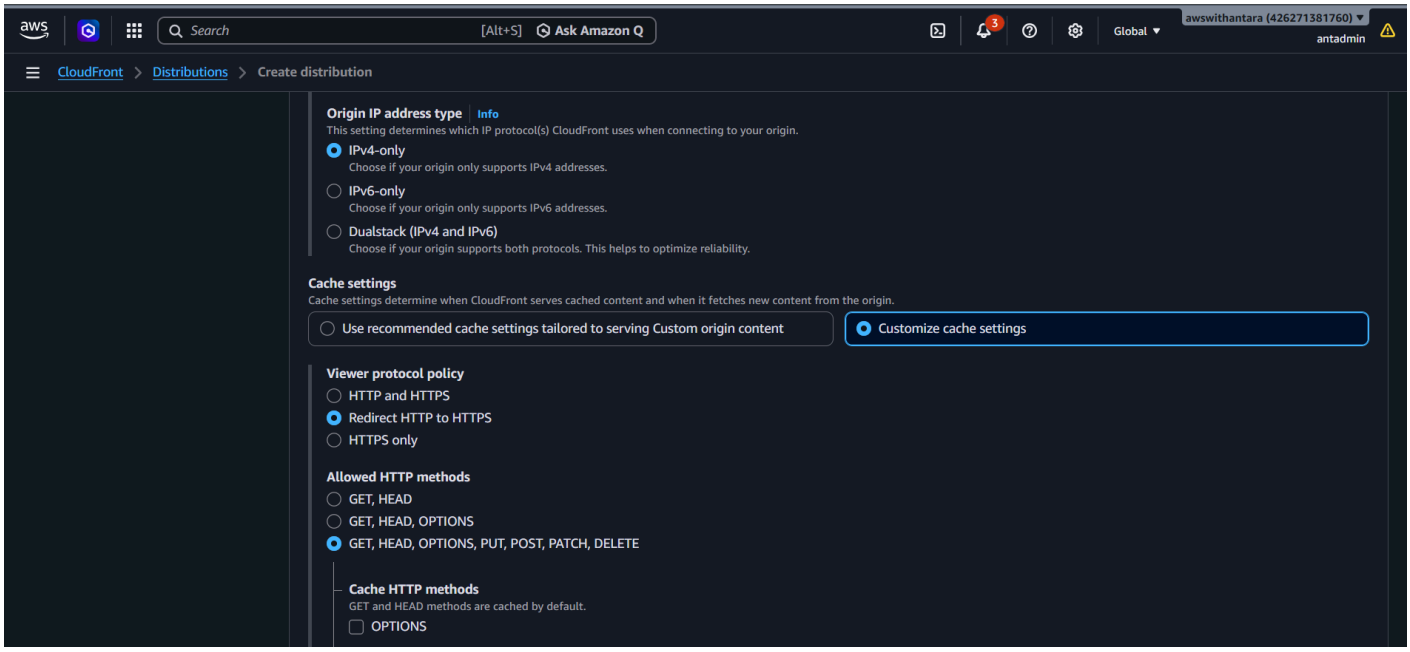


CloudFront Distribution Setup: Origin Type & Domain (EC2 public DNS)

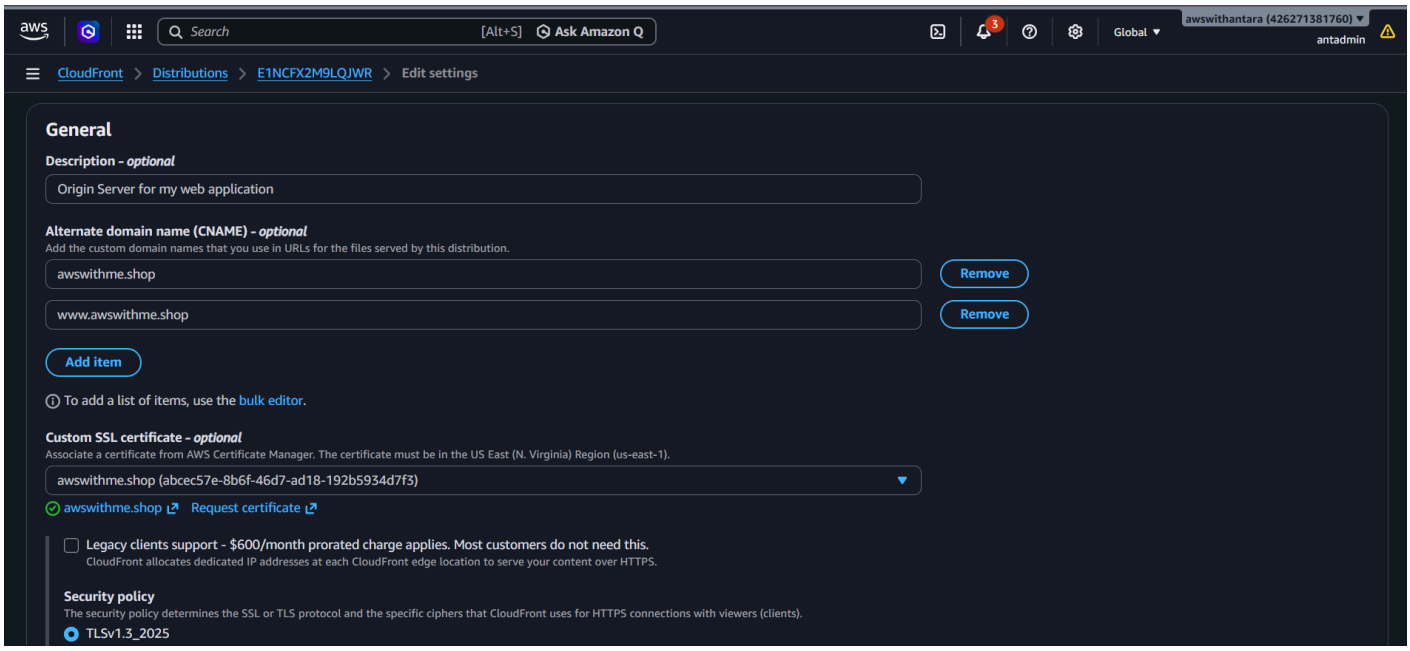


CloudFront Distribution Setup: Origin Protocol

- For Origin type, Other needs to be selected for EC2 as origin where public DNS of **myLinuxVM** is used as custom origin
- Protocol is selected as **HTTP only** since CloudFront connects to **myLinuxVM** over port 80



CloudFront Distribution Setup: Origin IP Address Type, Viewer Protocol Policy & HTTP Methods



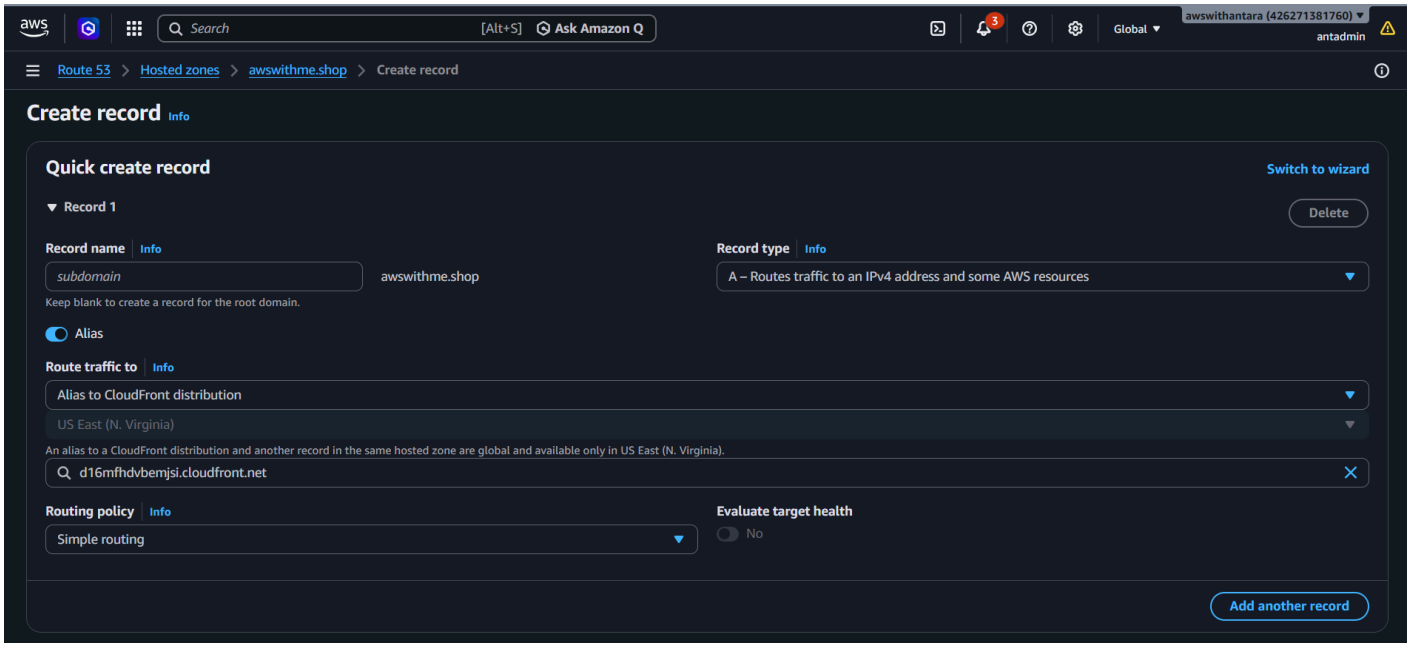
CloudFront Distribution Setup: Alternate Domain Names & Custom SSL Certificate

- The allowed HTTP methods are selected as **GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE** which is sufficient for static sites
- **Custom SSL certificate** must be added along with alternate domain names for SSL/TLS handshake

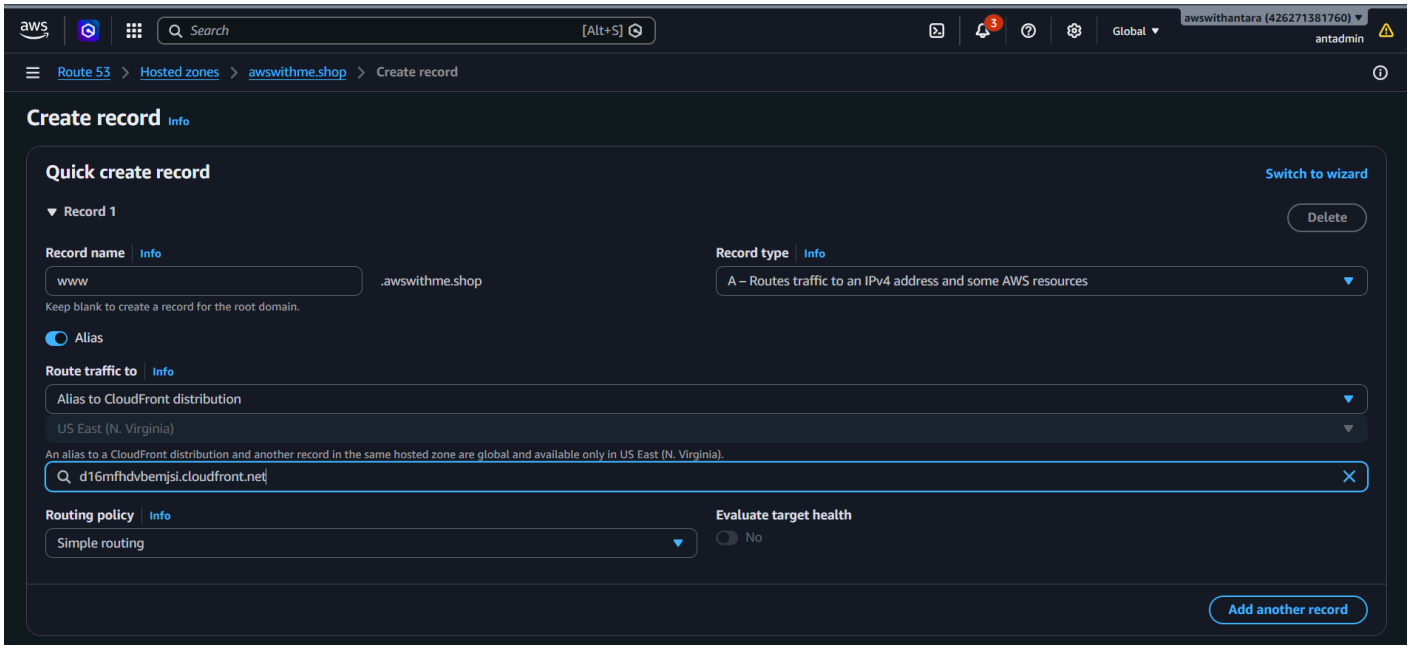
Phase 4: DNS Resolution (Amazon Route 53)
DNS records are created in the public hosted zone of the custom domain (**awswithme.shop)** in order to connect it to CloudFront DNS.

DNS Records Details:

- Record 1:
  - Record name: awswithme.shop
  - Record type: A
  - Route traffic to: Alias to CloudFront distribution
  - Routing policy: Simple routing
- Record 2:
  - Record name: www.awswithme.shop
  - Record type: A
  - Route traffic to: Alias to CloudFront distribution
  - Routing policy: Simple routing



DNS Records Creation in Route 53: Record Details for **awswithme.shop**



DNS Records Creation in Route 53: Record Details for **www.awswithme.shop**

Inference: The implementation of this edge-optimized architecture demonstrates a robust global content delivery. By successfully decoupling domain resolution (Route 53), edge caching (CloudFront), and origin compute (EC2), this architecture establishes a scalable foundation capable of handling enterprise-level traffic while maintaining stringent security postures.

- **Key Engineering Decisions:**
    - **Edge Caching (CloudFront)**: Implemented to drastically reduce latency by caching static assets at global Edge Locations, minimizing the compute load on the EC2 origin.
    - **Origin Protection (Prefix Lists):** Configured the EC2 Security Group to exclusively accept incoming HTTP traffic from the AWS Managed Prefix List for CloudFront. This prevents unwanted bypassing of the CDN security policies via direct IP routing.
    - **SSL Termination:** Terminated the TLS connection at the CloudFront edge using AWS ACM, offloading cryptographic processing from the EC2 compute instance while ensuring end-to-end encryption for the user.